





*Serviço Nacional
de Aprendizagem
Industrial*

Linguagem Java

Programação de Aplicativos

Linguagem Java

- **Instrutor:** Lucas Ribeiro Arêas
- **Unidade Curricular:** Programação de Aplicativos
- **Curso:** Técnico em Desenvolvimento de Sistemas

SENAI Vitória - DR/ES

Introdução

Introdução

Nesta disciplina estudaremos os conceitos de programação orientada a objetos. Para isso, precisaremos aprender a programar em uma linguagem de programação que siga esses conceitos: a linguagem **Java**.

1 - Funcionamento do Java

Funcionamento do Java

Um programa Java precisa passar por um processo de compilação para ser analisada a existência de erros de sintaxe. Esse processo de compilação traduz o código-fonte escrito pelo programador para uma linguagem intermediária chamada Java *bytecodes*.

Funcionamento do Java

Esse processo de tradução dos códigos fontes para Java *bytecodes* é feito por um programa chamado **compilador**. Então, é necessário que outra ferramenta chamada **interpretador** se responsabilize por interpretar esses *bytecodes* para o sistema operacional. Essa ferramenta que interpreta *bytecodes* é a máquina virtual Java (*Java Virtual Machine* - JVM).

2 - Compilando e Executando Códigos em Java

Compilando e Executando Códigos em Java

O conjunto de ferramentas necessárias para desenvolver, compilar e rodar aplicativos Java é disponibilizado em um kit conhecido como *Java Development Kit* (JDK). Assim, para começar a programar em Java você deve realizar o *download* do JDK e instalá-lo.

Compilando e Executando Códigos em Java

Com o JDK instalado, você pode começar a programar em Java utilizando um simples editor de texto para editar seus programas, como, por exemplo, o bloco de notas. Assim, você teria de editar seus programas, salvá-los com extensão *.java*, compilá-los e então executá-los.

Compilando e Executando Códigos em Java

Para facilitar e agilizar esse processo, existem disponíveis vários Ambientes de Desenvolvimento – *Integrated Development Environment* (IDE), que dão suporte à linguagem Java. Um IDE é um programa de computador que reúne ferramentas de apoio ao desenvolvimento de *software* com o objetivo principal de agilizar o processo de codificação.

3 - Ambientes Integrados de Desenvolvimento

Ambientes Integrados de Desenvolvimento

Dos diversos IDEs para programação em Java, os dois mais amplamente utilizados são o **NetBeans** e o **Eclipse**. Nessa disciplina utilizaremos o NetBeans.

Ambientes Integrados de Desenvolvimento

O NetBeans IDE é um ambiente de desenvolvimento integrado gratuito e de código aberto. Esse IDE é executado em muitas plataformas, como Windows, Linux, Solaris e MacOS, sendo fácil de instalar e usar.

Ambientes Integrados de Desenvolvimento

O NetBeans IDE oferece aos desenvolvedores todas as ferramentas necessárias para criar aplicativos profissionais de *desktop*, empresariais, *web* e móveis multiplataformas. Assim, todo o processo de edição, compilação e execução dos programas será feito dentro do NetBeans.

4 - Introdução a Programação em Java

Introdução a Programação em Java

Nesta etapa da unidade curricular ainda não trataremos sobre orientação a objetos, que é uma das principais características da linguagem Java. Neste momento focaremos em desenvolver programas utilizando a linguagem de programação Java.

Introdução a Programação em Java

Veja a seguir um exemplo que código simples em Java.

Introdução a Programação em Java

Exemplo em JAVA

```
public class Exemplo01 {  
  
    public static void main(String[] args) {  
        System.out.println("Primeiro Exemplo!");  
    }  
}
```

Introdução a Programação em Java

O exemplo que acabamos de ver é utilizado para imprimir a seguinte frase na tela: **Primeiro Exemplo!**

Introdução a Programação em Java

Cabe destacar um ponto importante antes de seguirmos com a análise do nosso exemplo, o Java é *case sensitive*, ou seja, o seu compilador diferencia letras minúsculas de maiúsculas. Logo, devemos ter muita atenção ao digitar!

Introdução a Programação em Java

A principal diferença que podemos notar já de início é que todo programa em Java inicia-se com a definição de uma classe. Uma classe é definida pela palavra reservada `class`, seguida pelo nome da classe (neste caso, o nome da classe é “**Exemplo01**”).

Introdução a Programação em Java

Por convenção, todo nome de classe inicia-se com uma letra maiúscula. Um programa Java é formado por uma ou mais classes. Não se preocupe agora com a definição de classe. Estudaremos esse conceito mais à frente, na parte de **Introdução a Orientação a Objetos**.

Introdução a Programação em Java

Todo programa Java tem sua execução iniciada pelo método ***main***, alguma das classes do programa Java deverá conter um método ***main***. A classe que contém o método ***main*** é chamada de classe principal.

Introdução a Programação em Java

Não se preocupe com alguns termos vistos agora, como por exemplo, as palavras “*public*” e “*static*” que aparecem na definição do método ***main***, elas serão explicadas em aulas futuras.

Introdução a Programação em Java

A palavra “*void*” que aparece no exemplo visto anteriormente indica que o método ***main*** da classe **Exemplo01** não possui retorno, ou seja, seu retorno é “vazio”.

Introdução a Programação em Java

O argumento “String args[]” é um vetor de Strings formado por todos os argumentos passados ao programa na linha de comando quando o programa é invocado.

Introdução a Programação em Java

Já o comando “`System.out.println ( )`” é utilizado para imprimir algo na tela.

Introdução a Programação em Java

Você poderia editar esse programa no bloco de notas e salvá-lo com o nome “**Exemplo01.java**” e, em uma máquina com o *Java Development Kit* (JDK) instalado, compilá-lo digitando no prompt comando: “**javac Exemplo01.java**”.

Introdução a Programação em Java

Esse comando geraria o arquivo *bytecode* com o nome "**Exemplo01.class**". Então, para executar o programa, deveria ser digitado no *prompt* comando: "**java Exemplo01.class**".

Introdução a Programação em Java

Em Java, o nome do arquivo deve ser sempre igual ao nome da classe, seguida da extensão **“.java”**. No caso do nosso exemplo, o nome do arquivo precisa ser **“Exemplo01.java”** porque o nome da classe é **“Exemplo01”**.

Introdução a Programação em Java

Mas nas nossas aulas vamos utilizar o NetBeans, pois nos poupa bastante trabalho e seu editor oferece alguns recursos que facilitam a edição do código.

5 - Declarando Variáveis e Constantes

Declarando Variáveis e Constantes

Programas feitos em Java devem ter suas variáveis declaradas e inicializadas antes de serem utilizados. Em Java toda variável é identificada por um nome e tem um tipo definido no ato da declaração, que não pode ser alterado.

Declarando Variáveis e Constantes

Sintaxe para declaração de variáveis:

<tipo_da_variável> <nome_da_variável>;

Exemplos: `double salario_liquido, salario_bruto;`

Declarando Variáveis e Constantes

O tipo da variável define, além do tipo de dado que ela pode armazenar, o tamanho do espaço de memória que deve ser alocado para ela. Veja no quadro a seguir os tipos primitivos da linguagem Java.

Declarando Variáveis e Constantes

Tipos primitivos da linguagem Java

Tipo primitivo	Valores a serem armazenados	Tamanho em <i>bits</i>
<i>char</i>	Permite armazenar um caractere. Exemplo: 'a'.	16
<i>byte</i>	Permite armazenar um número inteiro de -128 até +127. Exemplo: 10.	8
<i>short</i>	Permite armazenar um número inteiro de -32.768 até +32.767. Exemplo: 10.	16
<i>int</i>	Permite armazenar um número inteiro de -2.147.483.648 até +2.147.483.647. Exemplo: 10.	32
<i>long</i>	Permite armazenar um número inteiro de -9.223.372.036.854.775.808 até +9.223.372.036.854.775.807. Exemplo: 10.	64
<i>float</i>	Permite armazenar um ponto flutuante de -3,40292347E+38 até +3,40292347E+38. Exemplo: 3.1416.	32
<i>double</i>	Permite armazenar um ponto flutuante de -1,79769313486231570E+308 até +1,79769313486231570E+308. Exemplo: 3.1416.	64
<i>boolean</i>	Permite armazenar apenas o valor <i>true</i> ou o valor <i>false</i> .	8

Declarando Variáveis e Constantes

Em Java não há a definição de constantes. Quando queremos definir variáveis com valores constantes em Java utilizamos a palavra reservada `final`.

Declarando Variáveis e Constantes

Sintaxe de declaração de constante em Java:

```
final <tipo_da_variável> <nome_da_variável> =  
<valor_constante>;
```

Exemplo: `final float PI = 3.1416;`

6 - Conversões Entre Tipos Primitivos

Conversões Entre Tipos Primitivos

A conversão de tipos consiste em utilizar uma variável ou valor de um tipo para gerar um novo valor de outro tipo. Em alguns casos essa conversão é feita de forma direta, como, por exemplo, quando atribuímos um valor inteiro a uma variável do tipo *double*.

Conversões Entre Tipos Primitivos

Nesse caso a conversão é direta (também chamada de conversão implícita) porque é óbvio que um número inteiro pode ser representado como um número real.

Conversões Entre Tipos Primitivos

Mas, alguns tipos de variáveis apresentam valores incompatíveis entre si. Assim, nesses casos não é possível fazer uma atribuição direta.

Conversões Entre Tipos Primitivos

Por exemplo, se tentarmos atribuir a uma variável inteira o valor de uma variável *double*, teremos um erro de compilação.

Conversões Entre Tipos Primitivos

```
double p = 23.3;  
int i;  
i=p; //não compila  
i=2.3; //também não compila
```

Conversões Entre Tipos Primitivos

Para que atribuições como esta sejam possíveis, precisamos solicitar explicitamente que o número real seja moldado (*casted*) como um número inteiro. Essa operação de conversão entre tipos é também chamada de *casting*. Para realizar a conversão explícita, coloca-se, antes da expressão a ser convertida, o tipo destino da transformação entre parênteses.

Conversões Entre Tipos Primitivos

```
double p = 23.3;  
int i;  
i = (int)p;    //agora compila  
i = (int)2.3;  //agora compila
```


Conversões Entre Tipos Primitivos

No primeiro caso do exemplo apresentado no lside anterior, o valor da variável *p* foi convertido para caber em uma variável inteira. Assim, a variável *i* assumiu o valor 23. Logo depois, a variável *i* assumiu o valor 2.

Conversões Entre Tipos Primitivos

Ao fazer o *casting* para forçar a atribuição de um valor a uma variável, deve-se prestar atenção aos limites de valores de cada tipo primitivo. Esses limites foram apresentados no quadro sobre os Tipos Primitivos apresentados no slide 40 desta apresentação.

Conversões Entre Tipos Primitivos

Por exemplo, se atribuimos o valor 200 a uma variável inteira e depois utilizamos um *casting* para atribuir esse valor a uma variável *byte* cujos possíveis valores são de -128 a 127 não teremos erro de compilação, mas, como 200 estoura o valor máximo para uma variável *byte*, ocorrerá uma falha conhecida como *overflow* e o valor armazenado na variável *byte* será -56, conforme mostrado a seguir.

Conversões Entre Tipos Primitivos

```
int i = 200;  
byte c = (byte) i; //Compila normalmente  
System.out.println("c = " + c); // O valor impresso será -56
```

Conversões Entre Tipos Primitivos

As conversões entre tipos primitivos podem ainda ser necessárias em expressões aritméticas quando são utilizados operandos com tipos divergentes. Veja o exemplo no próximo slide.

Conversões Entre Tipos Primitivos

```
long l = 12;
int i1 = 15;
int i2;
✗ i2 = i1 + l; //Não compila pois, o resultado da soma será um long
i2 = i1 + (int)l; //Agora compila
byte b1, b2 = 10;
✗ b2 = b1 + (byte)i1; //Mesmo com casting não compila!!!!
```

Conversões Entre Tipos Primitivos

Observe que no exemplo que acabamos de ver foi necessário fazer a conversão da variável *l* que é do tipo *long* para o tipo *int*, pois quando é realizada uma operação entre operadores de tipos distintos o Java faz as contas utilizando o maior tipo que aparece nas operações, no caso, *long*.

Conversões Entre Tipos Primitivos

No entanto, na última linha do exemplo, mesmo fazendo a conversão da variável *i1* que é do tipo *int* para o tipo *byte* continuou não compilando. Isso aconteceu porque o Java armazena o resultado de expressões em, no mínimo, um *int*.

Conversões Entre Tipos Primitivos

Assim, para que o trecho de código apresentado no slide 56 seja compilado sem erros, as duas linhas marcadas devem ser retiradas ou comentadas.

Conversões Entre Tipos Primitivos

Conversão entre tipos

Tipo de Origem	Tipo de Destino	Exemplo
<i>int</i>	<i>double</i>	<i>int a = 20;</i> <i>double b = a; //nesse caso a conversão é implícita</i>
<i>double</i>	<i>int</i>	<i>double a = 20.1;</i> <i>int b = (int) a; //conversão explícita</i>
<i>double</i>	<i>float</i>	<i>double a = 20.1;</i> <i>float b = (float) a; //conversão explícita</i>
<i>long</i>	<i>int</i>	<i>long a = 20;</i> <i>int b = (long) a; //conversão explícita</i>

Conversões Entre Tipos Primitivos

Cabe destacar que valores do tipo boolean não podem ser convertidos para nenhum outro tipo.

7 - Atribuição, Entrada e Saída de Dados em Java

Atribuição, Entrada e Saída de Dados em Java

Em Java, quando queremos atribuir um determinado valor a uma variável, utilizamos o operador “=”.

Atribuição, Entrada e Saída de Dados em Java

Para obter entradas de dados digitados pelo usuário em Java utilizaremos a classe ***Scanner***. O uso dessa classe é bastante simples e poderemos entender melhor no exemplo apresentado a seguir.

Atribuição, Entrada e Saída de Dados em Java

```
package pacotel;  
import java.util.Scanner;  
public class ExemploUsandoClasseScanner {  
    public static void main(String[] args) {  
        //File fil = new File("c:/arq.txt");  
        Scanner scan = new Scanner(System.in);  
        int num2 = scan.nextInt();  
        int num1 = scan.nextInt();  
        double num3 = scan.nextDouble();  
        System.out.print(num2+num1);  
        System.out.println(num3);  
        scan.close();  
    }  
}
```

Atribuição, Entrada e Saída de Dados em Java

Ainda não aprendemos o conceito de classe, mas utilizaremos algumas classes básicas, como a classe ***Scanner***. O primeiro passo é criarmos um objeto do tipo ***Scanner***, conforme a linha:

```
Scanner scan = new Scanner(System.in);
```


Atribuição, Entrada e Saída de Dados em Java

Para lermos valores do tipo *int* utilizando o objeto *scan*, utilizamos o método *nextInt()*. Já para valores do tipo *double*, utilizamos o método *nextDouble()*. Não podemos esquecer de fechar o objeto quando findamos seu uso pelo método *close()*.

Atribuição, Entrada e Saída de Dados em Java

Para exibir mensagens e dados na tela utilizaremos em Java os “métodos”

System.out.print e *System.out.println*. A única diferença entre eles é que o primeiro imprime a mensagem na mesma linha em que a mensagem anterior foi impressa, enquanto o segundo imprime a mensagem em uma nova linha.

8 - Operadores

Operadores

Os operadores aritméticos são símbolos que representam operações aritméticas, ou seja, as operações matemáticas básicas. Os operadores aritméticos da linguagem Java podem ser visualizados no Quadro a seguir.

Operadores

Operadores aritméticos da linguagem Java

Operador	Descrição da operação matemática
+	Soma (inteira e ponto flutuante)
-	Subtração (inteira e ponto flutuante)
*	Multiplicação (inteira e ponto flutuante)
/	Divisão (inteira e ponto flutuante)
--	Decremento unário (inteiro e ponto flutuante)
++	Incremento unário (inteiro e ponto flutuante)
%	Resto da divisão de inteiros

Operadores

Assim como os operadores aritméticos, em java também temos os operadores relacionais e os operadores lógicos. Estes outros são utilizados para realizar comparações e proposições lógicas. Vejamos a seguir esas outras duas categorias de operadores em Java.

Operadores

Os operadores relacionais são utilizados para realizar comparações entre dois valores de um mesmo tipo, retornando como resultado sempre um valor lógico, ou seja, verdadeiro ou falso. Vejamos a seguir um quadro com os operadores relacionais do Java.

Operadores

Operadores relacionais de Java

Descrição	Operador
igual a	== (dois sinais de igual)
maior que	>
menor que	<
maior ou igual a	>=
menor ou igual a	<=
diferente de	!=

Operadores

Os operadores lógicos são utilizados para formar novas proposições lógicas a partir da junção de duas outras. O quadro a seguir mostra os operadores lógicos do Java.

Operadores

Operadores lógicos de Java

Descrição	Operador
E	&&
OU	(duas barras verticais)
NÃO	! (exclamação)

9 - Estruturas de Decisão

Estruturas de Decisão

O Java fornece três tipos de instruções de seleção/decisão. A instrução ***if*** realiza uma ação se uma condição for verdadeira ou pula a ação se ela for falsa. A instrução ***if...else*** realiza uma ação se uma condição for verdadeira ou realiza uma ação diferente se a condição for falsa. A instrução ***switch*** realiza uma de muitas ações, dependendo do valor de uma expressão.

Estruturas de Decisão

O exemplo apresentado a seguir ilustra a utilização do comando ***if... else***. Nesse exemplo, lemos uma nota e verificamos se ela está entre 0 e 100: se estiver, a nota é considerada válida; caso contrário, inválida.

Estruturas de Decisão

```
package pacote1;
import java.util.Scanner;
public class ExemploIf {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.println("Entre com a nota: ");
        double nota = scan.nextDouble();
        if (nota <= 100 && nota >= 0) {
            System.out.println("Nota = " + nota + " ( valor válido )");
        } else {
            System.out.println("Nota = " + nota + " ( valor inválido )");
        }
    }
}
```

Estruturas de Decisão

O próximo exemplo que iremos ver é o sobre o uso do comando ***switch-case***.

Estruturas de Decisão

```
package pacote1;
public class ExemploSwitchCase {
    public static void main(String[] args) {
        int mes;
        mes = Integer.parseInt(args[0]);
        switch (mes) {
            case 1: System.out.println("Janeiro"); break;
            case 2: System.out.println("Fevereiro"); break;
            case 3: System.out.println("Março"); break;
            case 4: System.out.println("Abril"); break;
            case 5: System.out.println("Maio"); break;
            case 6: System.out.println("Junho"); break;
            case 7: System.out.println("Julho"); break;
            case 8: System.out.println("Agosto"); break;
            case 9: System.out.println("Setembro"); break;
            case 10: System.out.println("Outubro"); break;
            case 11: System.out.println("Novembro"); break;
            case 12: System.out.println("Dezembro"); break;
            default: System.out.println("Mes inválido");
        }
    }
}
```


10 - Estruturas (Laços) de Repetição

Estruturas (Laços) de Repetição

O Java fornece três instruções de repetição que permitem que programas executem instruções repetidamente, contanto que uma condição permaneça verdadeira. As instruções de repetição são ***while***, ***do...while*** e ***for***.

Estruturas (Laços) de Repetição

Vejam os a seguir um exemplo de uso do comando ***for***. Neste comando nós declaramos e inicializamos a variável que servirá como contadora para o *loop* dentro da própria estrutura do comando. Na estrutura do ***for*** nós também definimos o critério de parada e a forma de incremento ou decremento da variável contadora.

Estruturas (Laços) de Repetição

```
package pacote1;
public class ExemploFor {
    public static void main(String[] args) {
        for( int i = 0; i <= 10; i++)
        {
            System.out.print(i + " ");
        }
        System.out.println();
        System.out.println("Acabou");
    }
}
```

Estruturas (Laços) de Repetição

A seguir veremos exemplos de código dos comando ***while (enquanto)*** e ***do...while (faça - enquanto)***. A diferença básica entre eles é que o ***while*** verifica antes de entrar no *loop* de execução. Já o ***do...while*** verifica ao final da execução do laço de repetição. Logo, na estrutura do ***do...while*** a execução do bloco interno ocorrerá pelo menos uma vez.

Estruturas (Laços) de Repetição

```
package pacote1;  
public class ExemploWhile {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i <= 10) {  
            System.out.print(i + " ");  
            i++;  
        }  
        System.out.println();  
        System.out.println("Acabou");  
    }  
}
```

Estruturas (Laços) de Repetição

```
package pacote1;  
public class ExemploDoWhile {  
    public static void main(String[] args) {  
        int i = 0;  
        do{  
            System.out.print(i + " ");  
            i++;  
        }while (i <= 10);  
        System.out.println();  
        System.out.println("Acabou");  
    }  
}
```

11 - Escopo de Variáveis

Escopo de Variáveis

Na linguagem Java podemos declarar variáveis a qualquer momento. Entretanto, o trecho de código em que a variável será válida dependerá de onde ela foi declarada.

Escopo de Variáveis

Quando abrimos um novo bloco com as chaves, as variáveis declaradas ali dentro só existirão até o fim daquele bloco, ou seja, até o ponto onde se fecham as chaves do bloco.

Escopo de Variáveis

Vejamos um exemplo a seguir.

Escopo de Variáveis

```
public class ExemploAulaJava {  
    public static void main(String[] args) {  
        // Aqui a variável x ainda não existe  
        int x = 1;  
        // Aqui a variável x já existe  
        while ( x > 5 ) {  
            // Aqui a variável x continua existindo e y ainda não existe  
            int y = 0;  
            // Aqui a variável y já existe  
        }  
        // Aqui a variável já não existe mais. O x continua existindo  
    }  
}
```

Escopo de Variáveis

Tenha cuidado! Se criar uma variável dentro de um bloco e tentar utilizá-la fora desse bloco, haverá um erro de compilação.

12 - Vetores e Matrizes

Vetores e Matrizes

Como já aprendemos nas disciplinas anteriores, vetor (*array*) é uma estrutura de dados utilizada para representar certa quantidade de variáveis de valores homogêneos, ou seja, um conjunto de variáveis, todas do mesmo tipo. Em Java podemos criar vetores de tipos primitivos ou de objetos.

Vetores e Matrizes

Vejam os a seguir a declaração de um vetores em Java.

Vetores e Matrizes

Declaração:

```
<tipo_do_dado> <nome_do_vetor> [ ] = new  
<tipo_do_dado>[quantidade];
```

Exemplo: **int notas[] = new int[30];**

Vetores e Matrizes

No exemplo que acabamos de ver, será criado um vetor de 30 inteiros (inicializados com o valor zero).

Vetores e Matrizes

Também é possível inicializar um vetor com um conjunto de valores ao mesmo tempo em que o declaramos. Veja no exemplos a seguir:

```
int notas[ ] = { 1,2,3};
```

Vetores e Matrizes

Uma vez que o vetor já foi devidamente criado ele já pode ser utilizado. O acesso de cada elemento do vetor ocorre pelo seu índice (os índices de um vetor se iniciam do zero).

Vetores e Matrizes

A seguir veremos um exemplo de código de uso de vetores em Java.

Vetores e Matrizes

```
package exemplos;
import java.util.Scanner;

public class ExemploVetor {

    public static void main(String[] args) {
        float notas[];//Declaro o vetor mas ainda não aloco!
        /*Na próxima linha vou solicitar que o usuário informe a quantidade de
        * dados, lendo essa quantidade para a variável qtd.
        */
        System.out.println("Informe a quantidade de notas a serem digitadas:");
        Scanner scan = new Scanner(System.in);
        int qtd = scan.nextInt();
        //Agora que sei a quantidade vou alocar o vetor do tamanho desejado
        notas = new float[qtd];
        /*Vou fazer um laço para ler todas as notas. Note que usei o atributo
        * length como condição de parada do laço.
        */
        for (int i=0; i<notas.length;i++){
            System.out.println("Digite a nota do aluno numero " + i);
            notas[i]=scan.nextFloat();
        }
        /* Abaixo temos um laço que roda todo o vetor de notas imprimindo na
        * tela todas as notas atribuídas aos alunos.
        */
        for (int i=0; i<notas.length;i++){
            System.out.println("Nota do aluno numero " + i + " foi " + notas[i]);
        }
    }
}
```

Vetores e Matrizes

Vetores podem ter mais de uma dimensão, sendo conhecidos como vetores multidimensionais ou matrizes. As sintaxes para declaração e uso de matrizes são idênticas às de vetores acrescentando valores para cada uma de suas dimensões. A seguir um exemplo de declaração de matrizes em Java.

Vetores e Matrizes

```
int matriz[ ][ ] = new int[10][10];  
matriz[9][9] = -3;
```


Vetores e Matrizes

A partir do momento em que um vetor ou matriz foi criado, ele não pode mudar de tamanho. Se você precisar de mais espaço, será necessário criar um novo e copiar os elementos do vetor antigo para o novo.

13 - Boas Práticas na Programação em Java

Boas Práticas na Programação em Java

- **Classes e interfaces:** utilizam PascalCase. Exemplo: GerenciadorDeUsuarios, CalculadoraFinanceira.
- **Variáveis e métodos:** utilizam o camelCase, ou seja, começam com letra minúscula. Exemplo: calcularSalario(), nomeDoCliente.
- **Constantes:** são escritas em maiúsculas, separadas por underscores. Exemplo: TAXA_DE_JUROS.

Obrigado!